



北京邮电大学  
Beijing University of Posts and Telecommunications

## BUPT Compilers Lab1 2024Fall

Name: 项 枫

Student ID: 2022211570

Class Number: 2022211801

Container Number:

6eca9c2b53e9aaa0c65b6af6613814be05cc0d8ccc63b7df064c4a7230aab09f

## 目录

|     |                                                    |    |
|-----|----------------------------------------------------|----|
| 一、  | docker .....                                       | 3  |
| (一) | docker 简介 .....                                    | 3  |
| (二) | docker 安装 .....                                    | 3  |
| (三) | 实验环境配置 .....                                       | 7  |
| 二、  | Lab1.....                                          | 9  |
| (一) | Complete functions in the linked_list.c file ..... | 9  |
| (二) | Make the libll target .....                        | 12 |
| (三) | Run self-test examples in python3 .....            | 12 |
| 三、  | 总结 .....                                           | 12 |
| (一) | 总结.....                                            | 12 |
| (二) | 问题及解决.....                                         | 13 |

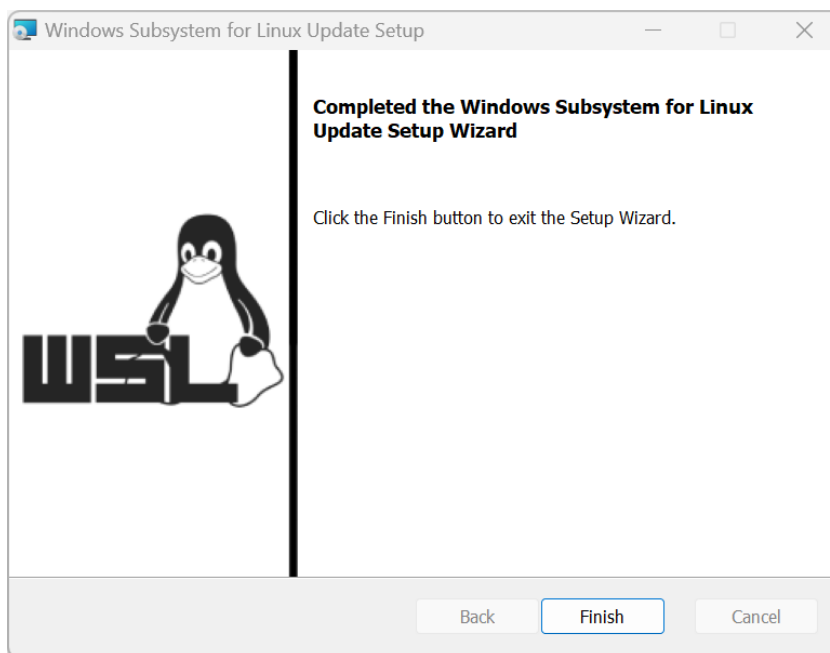
## 一、 docker

### (一) docker 简介

- 1、docker 是一个开源的应用容器引擎
- 2、诞生于 2013 年初，基于 Go 语言实现，dotCloud 公司出品（后改名 docker Inc）
- 3、docker 可以让开发者打包他们的应用以及依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的 Linux 机器上
- 4、容器是完全使用沙箱机制，相互隔离
- 5、容器性能开销极低
- 6、docker 可以运行在在 MAC、Windows、CentOS、UBUNTU 等操作系统上
- 7、官网：<https://www.docker.com>
- 8、在概念上类似于虚拟机但非常轻量，不会模拟底层硬件，只为每个应用提供完全隔离的运行环境，在环境中配置不同的软件
- 9、Image 镜像：可以理解为一个虚拟机快照，包含了部署的应用程序以及它所关联的所有库、软件，通过 Image 可以创建许多容器 Container
- 10、Container 容器：可以理解为运行起来的虚拟机，独立运行
- 11、dockerFile：自动化脚本，用来创建 Image，过程类似于在虚拟机中安装操作系统一样，只不过是一个自动化脚本

### (二) docker 安装

- 1、下载 WSL2 需要的 linux 内核 网址：<https://docs.microsoft.com/zh-cn/windows/WSL/WSL2-kernel>，点击下载最新的更新包，下好后双击运行安装即可



2、控制面板->程序->启用或关闭 Windows 功能->

勾选 适用于 Linux 的 Windows 子系统

勾选 虚拟机平台



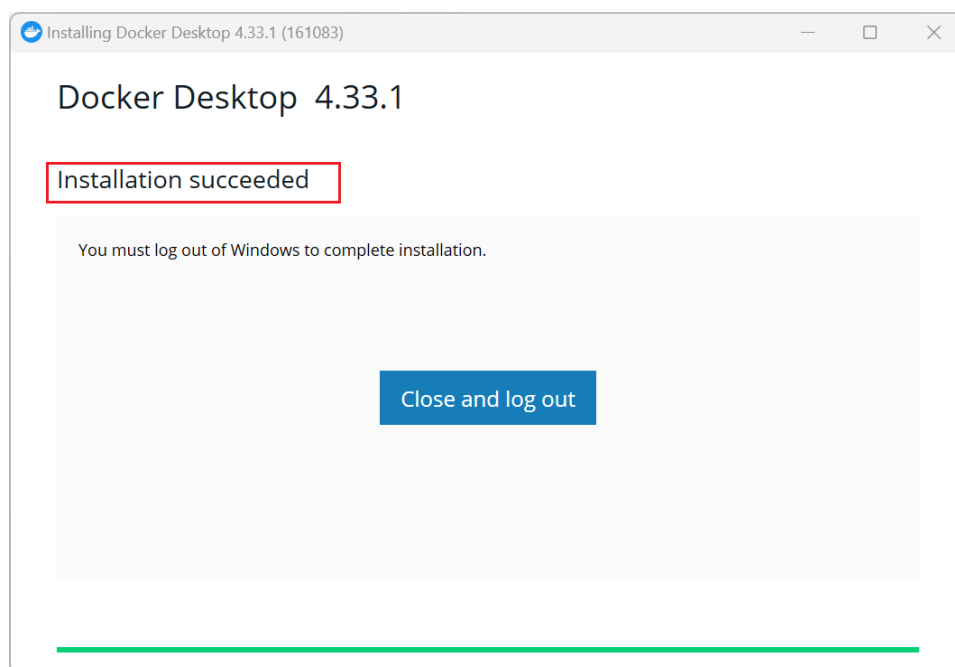
3、将 WSL2 设置为默认版本：CMD 运行 WSL --set-default-version 2

```

C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [版本 10.0.22631.4037]
(c) Microsoft Corporation. 保留所有权利。

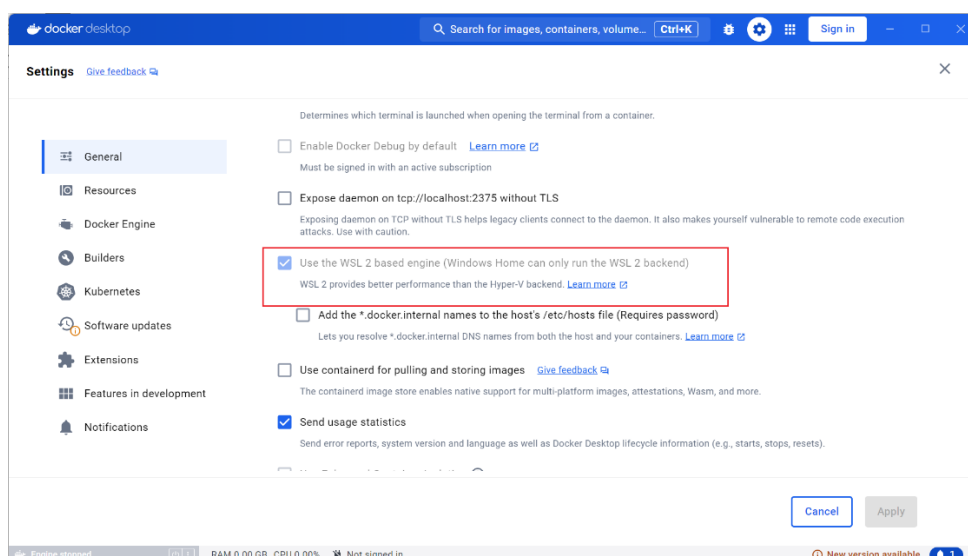
C:\Users\项枫>wsl --set-default-version 2
有关与 WSL 2 的主要区别的信息, 请访问 https://aka.ms/wsl2
操作成功完成。
C:\Users\项枫>
    
```

4、下载 docker 桌面版 <https://www.docker.com/products/docker-desktop>

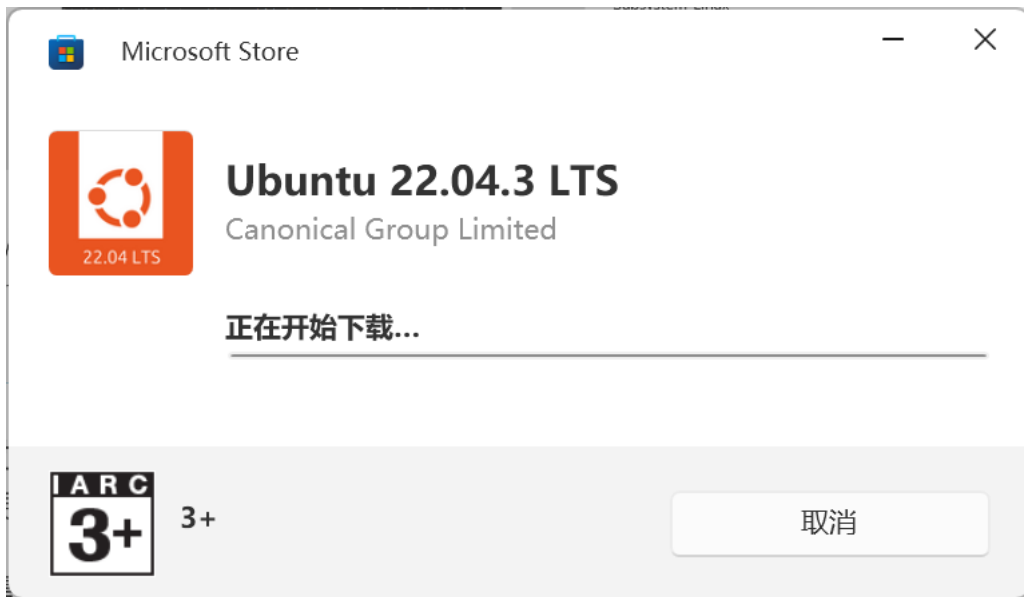


5、下好 docker-desktop 的安装包后直接双击运行安装即可

设置里勾选基于 WSL2 的引擎



6、下载 Ubuntu 22.04.3 LTS



```
feng@X86 ~
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: feng
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
适用于 Linux 的 Windows 子系统现已在 Microsoft Store!
你可以通过运行 "wsl.exe --update" 进行升级 或通过访问 https://aka.ms/wslstorepage
从Microsoft Store安装 WSL 将提供最新的 WSL 更新, faster.
有关详细信息, 请访问 https://aka.ms/wslstoreinfo<
>

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.10.16.3-microsoft-standard-WSL2 x86_64)

 * Documentation: https://help.ubuntu.com
 * Management: https://landscape.canonical.com
 * Support: https://ubuntu.com/advantage

This message is shown once a day. To disable it please create the
/home/feng/.hushlogin file.
feng@X86:~$
```

## 7、CMD 运行 WSL -l -v 以及 CMD 运行 docker 验证是否安装成功

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\项枫>WSL -l -v
NAME STATE VERSION
* Ubuntu-22.04 Running 2

C:\Users\项枫>docker
Usage: docker [OPTIONS] COMMAND

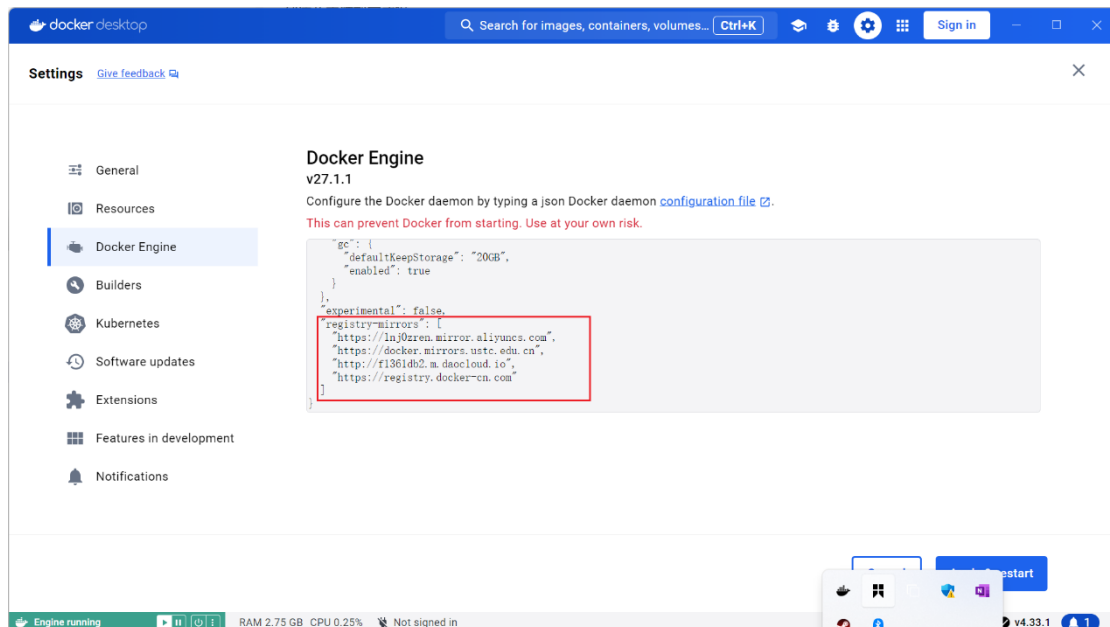
A self-sufficient runtime for containers

Common Commands:
run Create and run a new container from an image
exec Execute a command in a running container
ps List containers
build Build an image from a Dockerfile
pull Download an image from a registry
push Upload an image to a registry
images List images
login Log in to a registry
logout Log out from a registry
search Search Docker Hub for images
version Show the Docker version information
info Display system-wide information

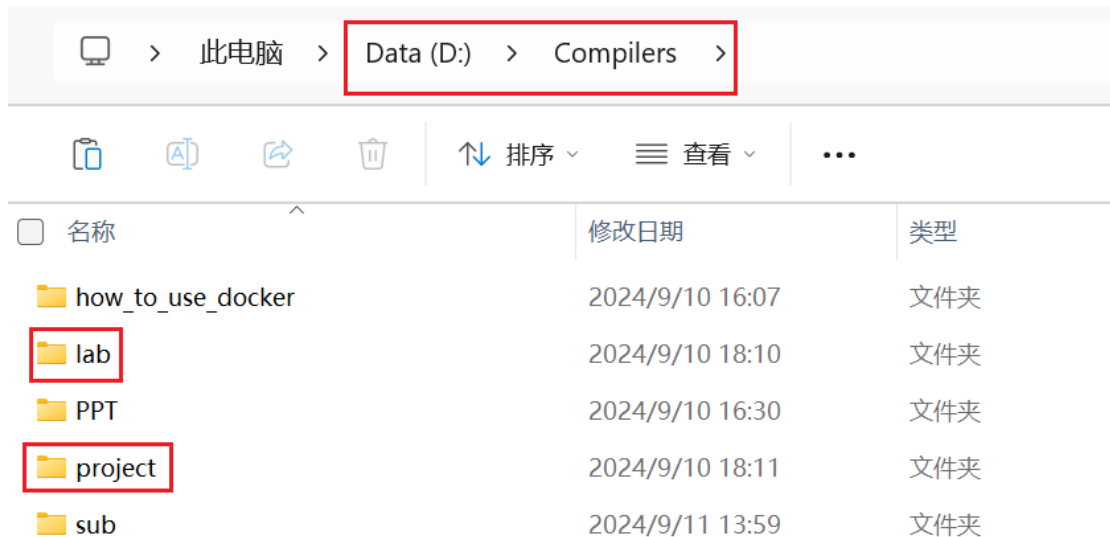
Management Commands:
builder Manage builds
buildx Docker Buildx
checkpoint Manage checkpoints
compose Docker Compose
container Manage containers
context Manage contexts
debug Get a shell into any image or container
```

### (三) 实验环境配置

1、安装 docker 后，由于国内从官方的 dockerHub 拉取镜像速度较慢，可以利用国内的镜像源



2、在本机创建一个文件夹专门存放 lab 及 project，路径中不要包含中文



3、编辑./docker\_script/docker-compose.yml，将替换为第一步中的路径

4、创建 image 并部署 container：进入 docker\_script 目录下，CMD 运行 docker-compose up -d

5、docker desktop 中启动部署好的 container，该 container 应该是 docker\_script 下的 compiler

[illegible]



7、compiler 中的 /mnt/Workspace 即为挂载的共享目录，lab 和 project 作业下载后放到主机的共享目录中

The terminal window shows the following commands and output:

```
PS D:\Compilers\how_to_use_docker\Windows\docker_script> docker exec -it compiler /bin/sh
# su

=====
root@6eca9c2b53e9:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@6eca9c2b53e9:/# cd mnt
root@6eca9c2b53e9:/mnt# ls
workspace
root@6eca9c2b53e9:/mnt# cd Workspace
root@6eca9c2b53e9:/mnt/Workspace# ls
lab
root@6eca9c2b53e9:/mnt/Workspace# cd lab
root@6eca9c2b53e9:/mnt/Workspace/lab# ls
Makefile README.pdf hello_world.c linked_list.c linked_list.h ll_test.py main.c report_photos
root@6eca9c2b53e9:/mnt/Workspace/lab/lab1#
```

The Windows File Explorer shows the directory path: Data (D:) > Compilers > lab > lab1 >. The files and folders listed are:

| 名称            | 修改日期            | 类型                    | 大小     |
|---------------|-----------------|-----------------------|--------|
| .idea         | 2024/9/10 16:03 | 文件夹                   |        |
| __pycache__   | 2024/9/10 15:54 | 文件夹                   |        |
| report_photos | 2024/9/10 22:18 | 文件夹                   |        |
| hello_world.c | 2023/5/19 18:36 | C Source File         | 1 KB   |
| linked_list.c | 2023/5/19 18:36 | C Source File         | 2 KB   |
| linked_list.h | 2023/5/19 18:36 | DevCpp.h              | 2 KB   |
| ll_test.py    | 2024/9/10 10:51 | JetBrains PyCharm ... | 5 KB   |
| main.c        | 2023/5/19 18:36 | C Source File         | 1 KB   |
| Makefile      | 2023/5/19 18:36 | 文件                    | 1 KB   |
| README.pdf    | 2024/9/10 11:00 | Adobe Acrobat 文档      | 296 KB |

## 二、 Lab1

### (一) Complete functions in the linked\_list.c file

```
/* my implementation goes here */

void linked_list_insert(node *head, int val, int index) {
    if (index < 0 || index > head->count) return; // Index out of bounds

    node *cur = head;
    node *new_node = (node*)malloc(sizeof(node));
    new_node->value = val;
```

```
// Traverse to the node just before the insertion point
for (int i = 0; i < index; i++) {
    cur = cur->next;
}

// Insert new node
new_node->next = cur->next;
cur->next = new_node;
head->count++;
}

void linked_list_delete(node *head, int index) {
    if (index < 0 || index >= head->count) return; // Index out of
    bounds

    node *cur = head;
    node *to_delete;

    // Traverse to the node just before the one to delete
    for (int i = 0; i < index; i++) {
        cur = cur->next;
    }

    // Remove node
    to_delete = cur->next;
    cur->next = to_delete->next;
    free(to_delete);
    head->count--;
}

void linked_list_remove(node *head, int val) {
    node *cur = head;
    node *to_delete;

    // Traverse until finding the node with the value to remove
    while (cur->next != NULL && cur->next->value != val) {
        cur = cur->next;
    }

    if (cur->next != NULL) {
        // Remove node
        to_delete = cur->next;
        cur->next = to_delete->next;
        free(to_delete);
    }
}
```

```
        head->count--;
    }
}

void linked_list_remove_all(node *head, int val) {
    node *cur = head;
    node *to_delete;

    // Traverse and remove all nodes with the value
    while (cur->next != NULL) {
        if (cur->next->value == val) {
            to_delete = cur->next;
            cur->next = to_delete->next;
            free(to_delete);
            head->count--;
        } else {
            cur = cur->next;
        }
    }
}

int linked_list_get(node *head, int index) {
    if (index < 0 || index >= head->count) return -0x80000000; // Use
    INT_MIN for out of bounds
    node *cur = head->next;
    for (int i = 0; i < index; i++) {
        cur = cur->next;
    }
    return cur->value;
}

int linked_list_search(node *head, int val) {
    node *cur = head->next;
    int index = 0;

    while (cur != NULL) {
        if (cur->value == val) {
            return index;
        }
        cur = cur->next;
        index++;
    }

    return -1; // Value not found
}
```

```
}

node *linked_list_search_all(node *head, int val) {
    node *result_list = linked_list_init();
    node *cur = head->next;
    int index = 0;

    while (cur != NULL) {
        if (cur->value == val) {
            linked_list_append(result_list, index); // Append the
index, not the value
        }
        cur = cur->next;
        index++;
    }

    return result_list;
}
```

## (二) Make the libll target

```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS D:\Compilers\how_to_use_docker\Windows\docker_script> docker exec -it compiler /bin/sh
# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
# cd mnt
# ls
Workspace
# cd Workspace
# ls
PPT how_to_use_docker lab project
# cd lab
# ls
lab1
# cd lab1
/bin/sh: 8: cd: can't cd to lab1
# cd lab1
# ls
Makefile README.pdf __pycache__ hello_world.c linked_list.c linked_list.h ll_test.py main.c report_photos
# make libll
gcc linked_list.c --shared -fPIC -o libll.so
#
```

## (三) Run self-test examples in python3

```
gcc linked_list.c --shared -fPIC -o libll.so
# python3 ll_test.py
python3: can't open file 'll_test.py': [Errno 2] No such file or directory
# python3 ll_test.py
.....
-----
Ran 8 tests in 0.002s
OK
#
```

## 三、总结

### (一) 总结

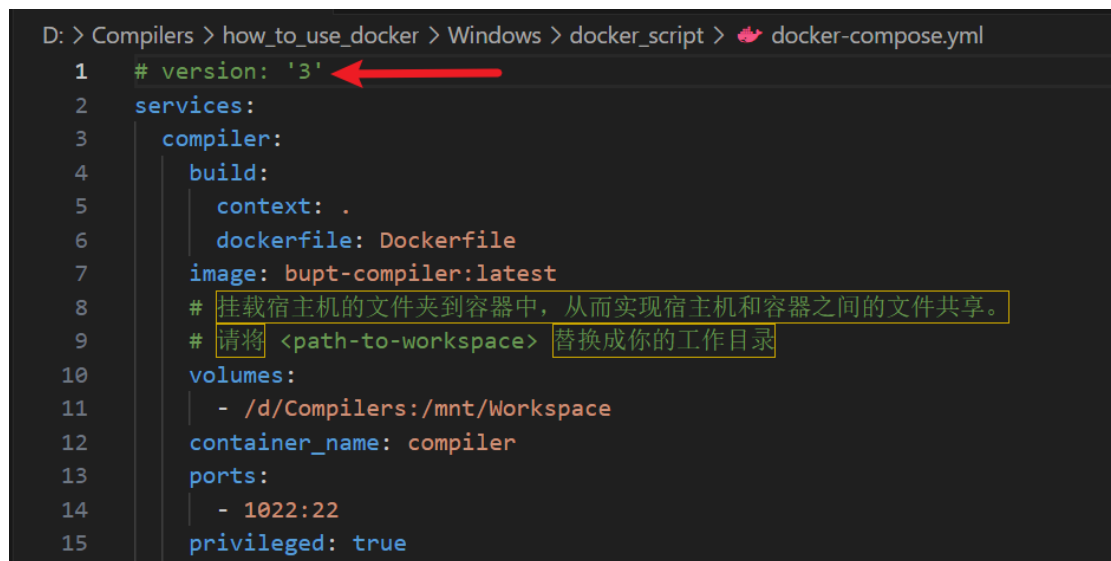
通过本次实验，掌握了 docker 的安装与使用并且回顾了链表的基本操作的代码复现。

## (二) 问题及解决

### 1、运行 docker-compose up -d 报错

报错：`version` is obsolete"

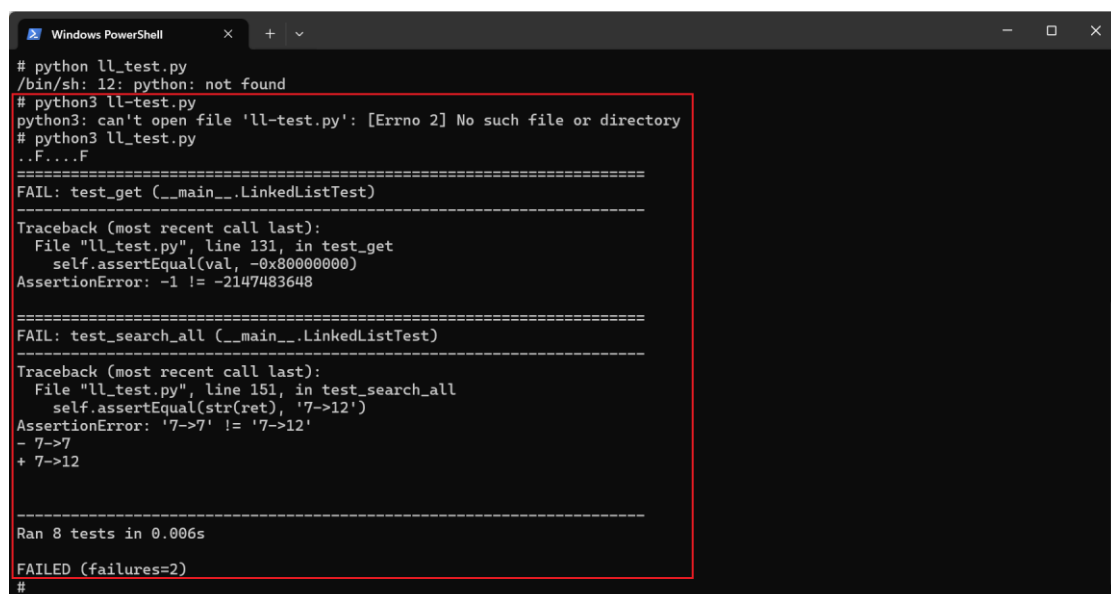
解决：将 version:3 注释，如下图。



```
D: > Compilers > how_to_use_docker > Windows > docker_script > docker-compose.yml
1  # version: '3'
2  services:
3    compiler:
4      build:
5        context: .
6        dockerfile: Dockerfile
7      image: bupt-compiler:latest
8      # 挂载宿主机的文件夹到容器中，从而实现宿主机和容器之间的文件共享。
9      # 请将 <path-to-workspace> 替换成你的工作目录
10     volumes:
11       - /d/Compilers:/mnt/Workspace
12     container_name: compiler
13     ports:
14       - 1022:22
15     privileged: true
```

### 2、运行 ll\_test.py 失败

报错：如下图。



```
Windows PowerShell
# python ll_test.py
/bin/sh: 12: python: not found
# python3 ll_test.py
python3: can't open file 'll_test.py': [Errno 2] No such file or directory
# python3 ll_test.py
..F...F
=====
FAIL: test_get (__main__.LinkedListTest)
=====
Traceback (most recent call last):
  File "ll_test.py", line 131, in test_get
    self.assertEqual(val, -0x80000000)
AssertionError: -1 != -2147483648
=====
FAIL: test_search_all (__main__.LinkedListTest)
=====
Traceback (most recent call last):
  File "ll_test.py", line 151, in test_search_all
    self.assertEqual(str(ret), '7->12')
AssertionError: '7->7' != '7->12'
- 7->7
+ 7->12
=====
Ran 8 tests in 0.006s
FAILED (failures=2)
#
```

解决：更改 `node *linked_list_search_all(node *head, int val)` 和 `int linked_list_get(node *head, int index)` 代码后，正常运行。